Buildings in the Desert

How 200-Year-Old Algebraic Geometry Is Revolutionizing Deep Learning

PART 3

The "Sheaf" in NeurIPS Papers: A Correction — What AI Actually Borrowed Was the Cellular Sheaf, Not the Étale Sheaf

A dialogue in seven parts · Tags: Deep Learning · Graph Neural Networks · Algebraic Topology · Sheaf Theory

Three-line summary (TL;DR)

1. "Sheaf" is appearing in more and more NeurIPS and ICML papers. But what AI actually borrowed is **not the étale sheaf** of algebraic geometry — it is the **cellular sheaf** from applied algebraic topology.
2. Cellular sheaves run on vectors and matrices, so they are differentiable in PyTorch. Étale sheaves run on the discrete action of Galois groups, so backpropagation does not apply. **This difference is the lifeline of AI implementations.**
3. Knowing this distinction precisely lets you read Sheaf GNN papers without being misled and translate them into code. It is a **quiet competitive advantage for an AI engineer.**

Introduction — Why you should get the word "sheaf" right

At NeurIPS 2022, Cristian Bodnar and collaborators (Michael Bronstein's group at Oxford) presented ***Neural Sheaf Diffusion***, a paper that used the mathematical theory of "sheaves" to dramatically improve graph neural networks on two of their most stubborn failure modes: **oversmoothing** and **heterophily**.

Since then, papers with **"Sheaf"** in the title have multiplied at NeurIPS and ICML. By 2025, follow-ups such as *Polynomial Neural Sheaf Diffusion (PolyNSD)* and *Sheaf HyperNetworks* are appearing in rapid succession.

But anyone trying to ride this wave runs into a first wall:

> **"Wait — when these papers say 'sheaf,' do they mean Grothendieck's étale sheaf?"**

Blog posts and short summaries often introduce "sheaves in AI" with the vocabulary of algebraic geometry: étale cohomology, Zariski topology, the absolute Galois group. Parts 1 and 2 of this series, in fact, did exactly the same thing: after explaining the étale topology we introduced the mathematical notion of a *sheaf* and described how it is being "built into AI models."

That framing — grand as it is, with its desert of finite-field buildings and its floating castle of Galois symmetry — carries a risk of **misleading** readers into believing that the sheaves inside a Sheaf GNN *are* étale sheaves.

They are not.

Open the official PyTorch implementation of Neural Sheaf Diffusion and you will find only real-valued vectors and matrix multiplications. There is not a single line of arithmetic over a finite field $\mathbb{F}_p$ and no Galois-group action anywhere.

This Part 3 is dedicated to clearing up that confusion. In a dialogue between two characters — a Young Data Scientist and an AI Research Scientist — we answer three questions:

- Which branch of mathematics did AI actually borrow its "sheaf" from?
- What do étale sheaves and cellular sheaves share, and where do they fundamentally part ways?
- Why is this knowledge a concrete asset for an AI engineer?

Part I — The "Sheaf" Inside AI

1.1 There is no Galois group in the PyTorch code

YOUNG DATA SCIENTIST

I pulled up the official implementation of *Neural Sheaf Diffusion*. It reads like pure PyTorch: tensor operations on nodes, linear layers, a sparse Laplacian. **I cannot see anywhere the code is computing with a Galois group**. Did I miss something?

AI RESEARCH SCIENTIST

You read it correctly. **Neural Sheaf Diffusion does not compute over any finite field**

$\mathbb{F}$_p and never invokes a Galois action. The evocative pictures from Parts 1 and 2 — buildings in the desert, the floating castle, laser beams from Gal($\mathbb{Q}/\mathbb{Q}$) — are there to transmit the *world view* of sheaf-like thinking: "stitch local data into a coherent global object." They are not a description of the actual algorithm.

AI RESEARCH SCIENTIST

The honest correspondence between that algebraic-geometry metaphor and the actual GNN looks like this:

Item	Algebraic-geometry metaphor (étale sheaf)	The actual GNN (cellular sheaf)
Underlying space	Prime spectrum Spec $\mathbb{Z}$	Graph of users, molecules, documents, ...
Data at a node	Solutions of an equation in $\mathbb{F}_p$	A real-valued vector in $\mathbb{R}^d$
"Translation" rule	Action of the Galois group (Frobenius)	A learnable matrix (linear map)

1.2 So why is it still allowed to call it a "sheaf"?

YOUNG DATA SCIENTIST

If what the two sides actually compute is so different, why do we use the *same word* — "sheaf" — for both?

AI RESEARCH SCIENTIST

Good question. The reason is that the étale sheaf of algebraic geometry and the cellular sheaf of GNNs **satisfy the same abstract axioms**. In mathematics, "sheaf" is not the name of a particular calculation; it is the name of any system that meets two rules:

1. Attach some kind of data to each place in a base space.
2. Provide "translators" (restriction maps) that make the data on overlapping places consistent where they meet.

AI RESEARCH SCIENTIST

Because that definition is axiomatic — an empty box waiting for concrete ingredients — different fields of mathematics fill the box with different contents, and each gets its own "flavor of sheaf."

Slot in the axiom	Étale sheaf	Cellular sheaf (GNN)
Space	Spec $\mathbb{Z}$	Graph G = (V, E)

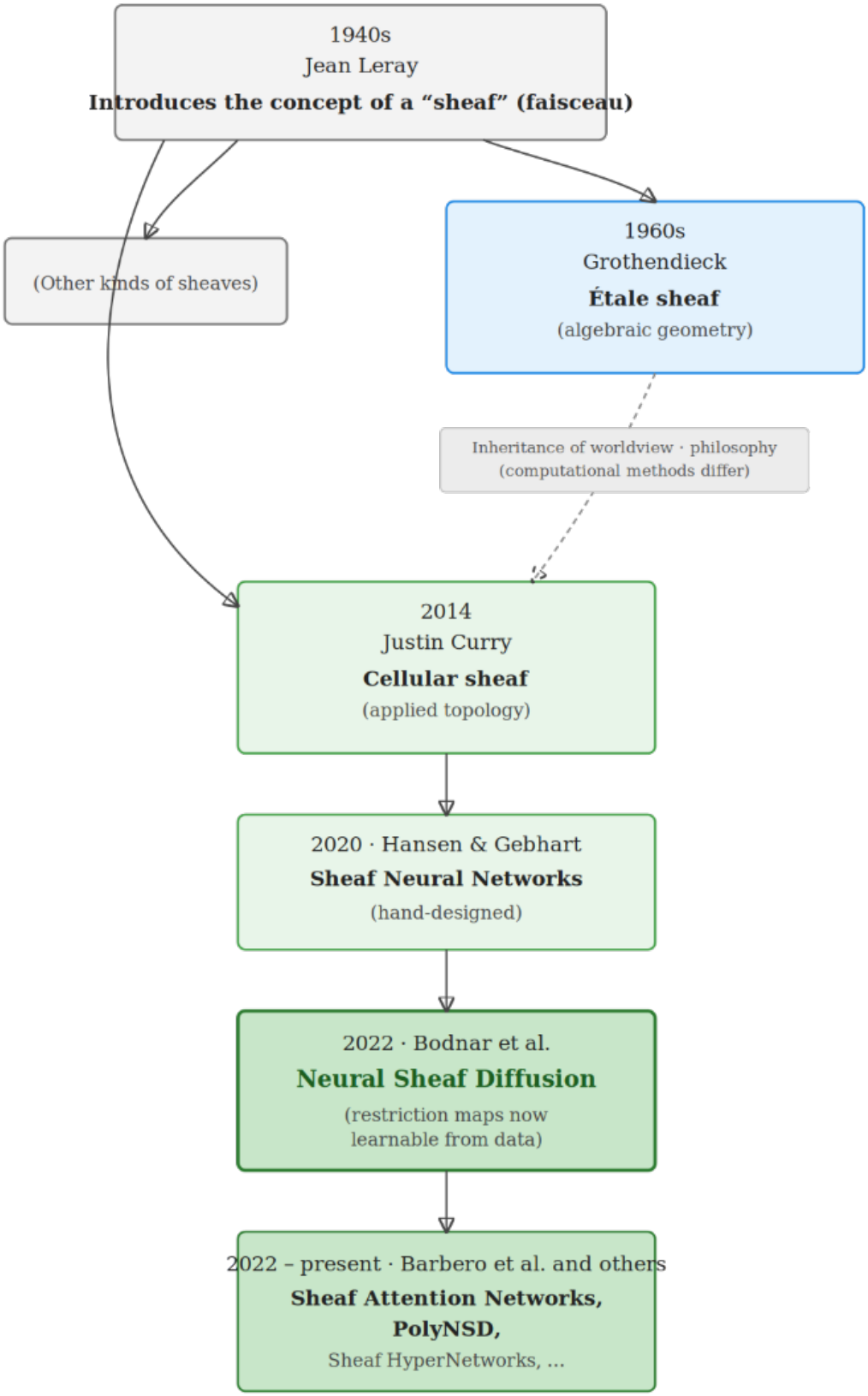
Slot in the axiom	Étale sheaf	Cellular sheaf (GNN)
Data at each place	Set of solutions to an equation	Vector space $\mathbb{R}^d$
Translator	Galois-group action	Matrix (linear map)

> **AI RESEARCH SCIENTIST**
>
> Bodnar and collaborators did not just *borrow the name "sheaf" by analogy* with algebraic geometry. They imported the **rigorous mathematical framework of cellular sheaves**, defined in applied algebraic topology, directly into a GNN architecture. That is why the word "sheaf" in the title is mathematically correct — it just points to a different sheaf.

1.3 The mathematical genealogy — cellular sheaf, not étale sheaf

This is the heart of the matter.

The mathematical lineage of the sheaf that AI imported is **not** "scheme theory → GNN." The correct lineage is:


1940s
Jean Leray
Introduces the concept of a “sheaf” (faisceau)
(Other kinds of sheaves)
1960s
Grothendieck
Étale sheaf
(algebraic geometry)
Inheritance of worldview · philosophy
(computational methods differ)
2014
Justin Curry
Cellular sheaf
(applied topology)
2020 · Hansen & Gebhart
Sheaf Neural Networks
(hand-designed)
2022 · Bodnar et al.
Neural Sheaf Diffusion
(restriction maps now
learnable from data)
2022 – present · Barbero et al. and others
Sheaf Attention Networks,
PolyNSD,
Sheaf HyperNetworks, ...

The concept of a **cellular sheaf** was systematized in 2014 by Justin Curry (then at Penn, now at SUNY Albany) in his PhD thesis *Sheaves, Cosheaves and Applications*, as a tool within applied algebraic topology. Together with Robert Ghrist's group, he showed that sheaf theory — historically a tool for continuous spaces — still has full force on *discrete* structures like graphs and cell complexes.

What AI researchers invented is **not** the concept of a cellular sheaf. It is the architectural idea of **making the cellular sheaf's translators — the restriction-map matrices — parameters that gradient descent can learn** from data.

Part II — Is the Étale-Sheaf Metaphor Mathematically Accurate?

2.1 "Desert buildings" and "sky castle" — fair as a picture of algebraic geometry

> **YOUNG DATA SCIENTIST**
>
> The metaphor of "buildings in the desert" and a "castle in the sky sending down laser beams" — is it actually accurate, *as a description of algebraic geometry*?

> **AI RESEARCH SCIENTIST**
>
> As a metaphor for algebraic geometry, yes — it is faithful. It captures the geometric intuition Grothendieck and modern arithmetic geometers use when they look at pure-algebraic equations:

- Each "desert building" corresponds to the fiber $\mathbb{F}_p$ of Spec $\mathbb{Z}$ over a prime ideal (p).
- "Ground-floor rooms" and "hidden upstairs rooms" correspond to $\mathbb{F}_p$-rational points and points in finite extensions $\mathbb{F}_{p^n}$.
- "Laser beams from the castle" correspond to the action of Frobenius elements from the absolute Galois group $Gal(\overline{\mathbb{Q}}/\mathbb{Q})$.

> **AI RESEARCH SCIENTIST**
>
> So the metaphor is not wrong as geometry. What it *does* risk hiding, when borrowed by AI writing, is that **the computation that metaphor describes is not the computation a GNN runs**.

2.2 Frobenius in action — a tiny puzzle

Let us do one concrete example to see how an étale sheaf actually works. Take the equation $x^2 + 1 = 0$ and two primes, $p = 5$ and $p = 3$.

Over $\mathbb{F}_5$ — solutions visible on the ground floor. The equation $x^2 + 1 \equiv 0 \pmod 5$ has the

two solutions $x = 2$ and $x = 3$ (since $2^2 = 4 \equiv -1$ and $3^2 = 9 \equiv -1$). Applying the Frobenius map $F(x) = x^5$ gives $2^5 \equiv 2$ and $3^5 \equiv 3$ (mod 5). **Each solution is fixed** — the roots are visible at the ground level.

Over $\mathbb{F}_3$ — solutions hidden in an extension. Over $\mathbb{F}_3$ the equation has no solutions; the solutions i and $-i$ live in the extension $\mathbb{F}_9$. Applying $F(x) = x^3$ we get $i^3 = -i$, so **Frobenius swaps i and −i indefinitely**. On the ground floor, nothing is visible; the Galois action lives entirely "upstairs."

This is the essence of an étale sheaf: **track the Frobenius permutation pattern of the solutions** — the action of a discrete group — rather than the solutions themselves. It was Grothendieck's key reframing.

> **And yet — this is exactly the kind of computation a GNN is not doing.** The vectors at the nodes of a GNN are not elements of a finite field; its restriction maps are not Frobenius morphisms. Conflate the two and you will misread Sheaf GNN papers at a basic level.

Part III — What Is a Cellular Sheaf, Concretely?

3.1 Definition

A cellular sheaf on a graph $G = (V, E)$ is specified by:

- For each node v, a vector space $\mathcal{F}(v) \cong \mathbb{R}^{d_v}$ — the **stalk** at that node;
- For each edge e, a vector space $\mathcal{F}(e)$;
- For every incidence "node v lies on edge e," a **restriction map** $\mathcal{F}_{v \to e} : \mathcal{F}(v) \to \mathcal{F}(e)$ — a plain matrix.

The **coboundary operator** δ is just a block matrix that stacks these restriction maps together. And the **sheaf Laplacian**

$$L_{\mathcal{F}} = \delta^T \delta$$

is a real, symmetric, positive-semidefinite matrix.

So the calculations that define a cellular sheaf reduce entirely to **concatenating vectors, multiplying them by matrices, transposing matrices, and computing products** — textbook linear algebra. This is exactly why the whole thing fits inside a PyTorch tensor graph: the operations are differentiable, their gradients flow, and SGD or Adam can train them end-to-end.

3.2 Étale vs cellular — the essential difference

Aspect	Étale sheaf	Cellular sheaf
Type of data	Groups or sets (Grp, Set)	Real vector spaces ($\mathrm{Vect}_{\mathbb{R}}$)
Translator	Galois-group action	Matrix (linear map)
Nature of the computation	Discrete · algebraic	Continuous · analytic
Differentiable?	No — backprop does not apply	Yes — trainable by gradient descent

> That single distinction — **"differentiable or not"** — is the reason cellular sheaves can be implemented in modern AI, and étale sheaves, so far, cannot.

Part IV — What Did the Sheaf GNN Actually Fix?

4.1 The fatal weaknesses of the vanilla GNN

A classical GNN such as GCN aggregates the features of neighboring nodes and averages them together:

$$x_A^{(new)} = x_A + x_B + x_C$$

Iterated enough times, every node converges to the same vector. This is **oversmoothing**.

Worse, on **heterophilic** graphs — where adjacent nodes tend to belong to *different* classes — the averaging step actively erases the very signal the model is supposed to detect, and accuracy collapses.

4.2 The fix: put a translator on every edge

A Sheaf GNN inserts a **learnable matrix** — a restriction map — between each pair of adjacent nodes. What the model now minimizes is not raw disagreement, but disagreement *after translation*:

$$disagreement = \mathcal{F}_{A \to e}(x_A) - \mathcal{F}_{B \to e}(x_B)$$

Summed and squared over all edges, this is exactly the sheaf Laplacian of Section 3, and the corresponding update rule is

$$X^{(t+1)} = \sigma(W X^{(t)} - \Delta_{\mathcal{F}} X^{(t)}).$$

> **The magic:** suppose the model learns, on one edge, the translator "flip the vector." Then node A holding v and node B holding $-v$ are "consistent" — once translated, they agree. So A and B can keep entirely different feature vectors while the sheaf registers *no contradiction*. Oversmoothing is avoided, and heterophily becomes a feature the model can *represent*, not a failure mode.

4.3 The Dirichlet-energy trick

The Dirichlet energy of a vanilla GNN,

$$\mathcal{E}(x) = \sum_{(i,j)} \| x_i - x_j \|^2,$$

is doomed to collapse to zero — which is precisely the mathematical signature of oversmoothing.

A Sheaf GNN replaces it with the sheaf Dirichlet energy:

$$\mathcal{E}_{\mathcal{F}}(x) = \sum_{e=(i,j)} \| \mathcal{F}_{i \to e}\, x_i - \mathcal{F}_{j \to e}\, x_j \|^2.$$

The condition for this energy to be zero is **not** $x_i = x_j$ but $\mathcal{F}_{i \to e}\, x_i = \mathcal{F}_{j \to e}\, x_j$. Equality after translation is enough; the nodes can still carry **wildly different vectors**. The energy can hit zero with the features of individual nodes remaining fully diverse.

Part V — Why This Is a Competitive Advantage for AI Engineers

---

5.1 You read papers correctly

When you pick up a paper with "Sheaf" in the title, you know immediately: **this is not about Galois groups; it is about matrix calculus over ℝ^d**. When the source code defines variables like `restriction_maps` or `sheaf_laplacian`, you recognize them as `nn.Parameter` matrices, not Frobenius morphisms. No confusion, no false depth.

5.2 You can make good implementation choices

How should the restriction maps be constrained? Orthogonal? Diagonal? General $d \times d$? Each choice trades accuracy against parameter count and inductive bias. You cannot make that trade rationally without a working mental model of *what a cellular sheaf is*.

5.3 You can ride the next research wave

As of 2025, Sheaf GNNs have branched out rapidly — federated learning (Sheaf HyperNetworks), algorithmic fairness (Sheaf Diffusion for Fairness), polynomial filters (PolyNSD), hypergraphs,

bundles. All of these sit on top of the cellular-sheaf framework. Getting the foundation right means you can absorb each new paper quickly instead of re-learning the vocabulary each time.

Part VI — Can an Étale Sheaf Be Built Directly into an AI?

YOUNG DATA SCIENTIST

Has anyone actually built an AI model that uses Grothendieck's étale sheaf *directly*?

AI RESEARCH SCIENTIST

Not as of today. The main obstacle is structural: **finite fields and Galois groups are discrete, and backpropagation needs differentiability**. Neural networks learn by moving through a continuous real-valued parameter space via gradient descent. In the world of $\mathbb{F}_5$, the number "2.001" simply does not exist. You cannot "nudge" a Frobenius morphism a little bit and read off a gradient.

AI RESEARCH SCIENTIST

That said, three indirect approaches are visible today:

3. **AI for Math.** Rather than embedding the étale sheaf inside a model, use AI as an *intuition assistant* for number theorists working on Galois representations. DeepMind's *Nature* paper on knot theory / representation theory is a well-known public example of this style.
4. **Neural networks over finite fields.** For privacy-preserving inference, one can build neural networks that run on $\mathbb{F}_p$ directly (CryptoNets, 2016 and successors). But note the sleight of hand: these networks are *trained* over $\mathbb{R}$ — the real numbers — and only the inference-time arithmetic is mapped to $\mathbb{F}_p$.
5. **Categorical deep learning.** Gavranović et al. (2024, ICML) rewrite every standard architecture — CNNs, RNNs, Transformers, GNNs — in the language of topoi, monads, and functors. This is a higher-level unification rather than a direct implementation of étale sheaves, but it is the most ambitious attempt to date to bring genuinely pure-mathematical structure into AI theory.

If anyone ever finds an algorithm that *truly* trains in the world of finite fields — with gradients, losses, and optimization all native to $\mathbb{F}_p$ — it will rewrite the history of deep learning.

Part VII — A Reading Map for Sheaf GNN Research

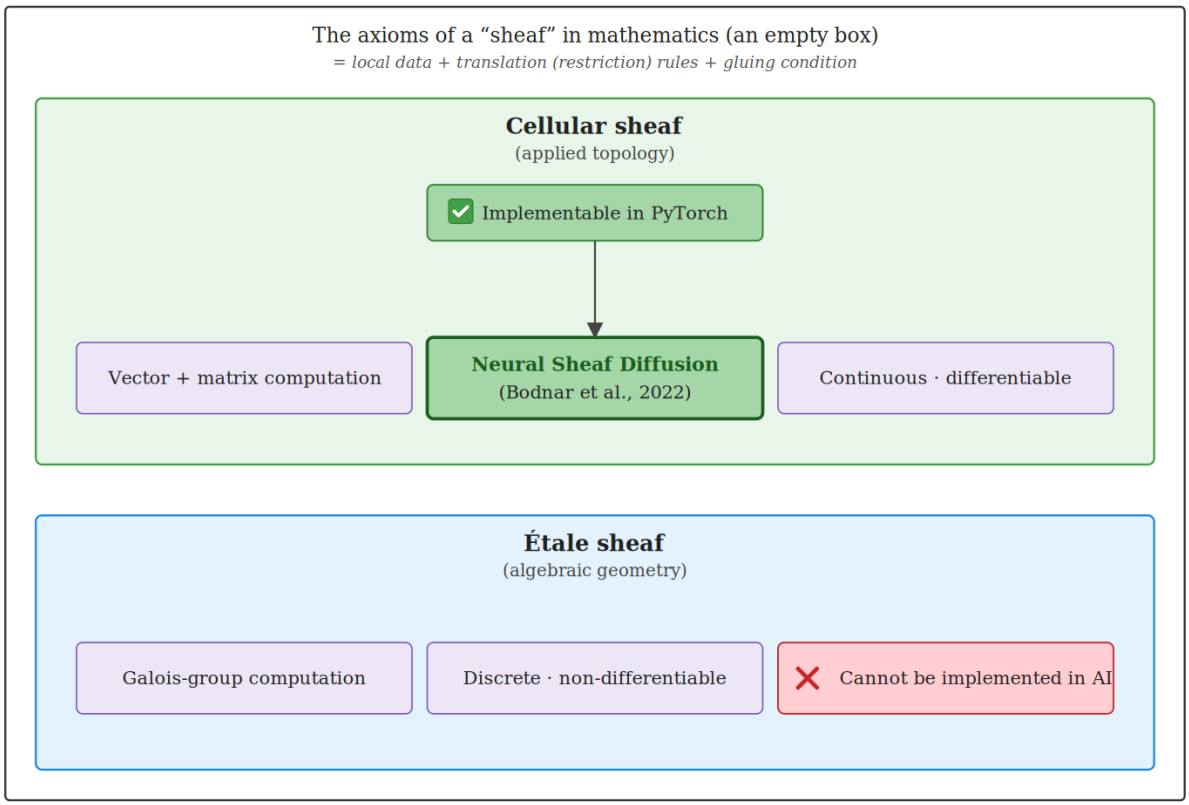
For readers who want to go beyond this article, here are four papers chosen to trace the evolution of the field:

#	Paper	Year	Role
1	Hansen & Ghrist, “Opinion Dynamics on Discourse Sheaves”	2021	Mathematical foundation: origin of the sheaf Laplacian.
2	Bodnar et al., “Neural Sheaf Diffusion” (NeurIPS)	2022	The canonical model: learnable restriction maps.
3	Barbero et al., “Sheaf Attention Networks” (NeurIPS Workshop)	2022	Evolution: Sheaf + Attention (generalizes GAT).
4	Bodnar et al., “Weisfeiler and Lehman Go Topological” (ICML)	2021	Theoretical backing: breaking past the WL test.

Recommended path

6. Use this article to internalize the big picture. ✓
7. Read §3 (sheaf Laplacian) and §4 (constraints on the restriction maps) of paper 2.
8. Clone the official GitHub repository, run a small example, and print tensor shapes as you trace the computation.
9. Move on to paper 3 to understand the fusion with attention mechanisms.

Summary



The axioms of "sheaf" are an empty box — local data plus translators plus a consistency rule. Two different fields of mathematics have filled that box in entirely different ways:

- **Étale sheaves** — from algebraic geometry. The translators are the action of a Galois group; the computation is discrete and not differentiable. **Cannot be implemented in a neural network.**
- **Cellular sheaves** — from applied algebraic topology. The translators are matrices; the computation is continuous and differentiable. **Implementable in PyTorch, trainable end-to-end.** This is what Bodnar et al. (2022) used.

> **The real accomplishment of Sheaf GNNs is to take a philosophical idea from étale sheaves — *"see the hidden connections between local pieces"* — and translate it into the one tool AI can actually compute with: learnable matrices.**

The accurate mathematical genealogy is: **sheaf theory on cell complexes → applied topology → graph neural networks**. Getting that one sentence right is the cleanest starting point for reading, implementing, and extending a Sheaf GNN paper.

References

1. C. Bodnar et al., "Neural Sheaf Diffusion: A Topological Perspective on Heterophily and Oversmoothing in GNNs," *NeurIPS*, 2022.
2. J. Hansen, R. Ghrist, "Opinion Dynamics on Discourse Sheaves," *SIAM J. Appl. Math.*, 2021.
3. F. Barbero et al., "Sheaf Attention Networks," *NeurIPS Workshop*, 2022.
4. C. Bodnar et al., "Weisfeiler and Lehman Go Topological," *ICML*, 2021.
5. J. Curry, "Sheaves, Cosheaves and Applications," Ph.D. Thesis, University of Pennsylvania, 2014.
6. J. Hansen, "Laplacians of Cellular Sheaves: Theory and Applications," Ph.D. Thesis, University of Pennsylvania, 2020.
7. T. Hansen, T. Gebhart, "Sheaf Neural Networks," *arXiv:2012.06333*, 2020.
8. R. Gilad-Bachrach et al., "CryptoNets: Applying Neural Networks to Encrypted Data," *ICML*, 2016.
9. J. Kileel et al., "On the Expressive Power of Deep Polynomial Neural Networks," *NeurIPS*, 2019.